

青岛科技大学

数据结构与算法分析

魏鹏超

19862239740

装
订
线

目 录	第一章 算法基础概念	$P_1 - P_5$
	第二章 递归算法	$P_5 - P_{10}$
	第三章 穷举法	$P_{11} - P_{15}$
	第四章 分治法	$P_{16} - P_{20}$
	第五章 回溯法	$P_{21} - P_{25}$
	第六章 分支界限法	$P_{26} - P_{30}$

第

页

青岛科技大学

第一章 算法基础概念

一、算法概述、设计

1. 算法是程序的灵魂

2. 程序 = 算法 + 数据结构

3. 算法是求解一个问题方法的描述

4. 算法的特性：有限性；正确性；可行性；输入性；输出性

5. 自然语言、流程图、伪代码、程序都可作为描述算法的方法

6. 算法设计的基本步骤：

① 分析求解问题；② 选择数据结构和算法设计策略

③ 描述算法；④ 证明正确性；⑤ 算法分析

二、算法设计工具 STL

1. STL 算法采用泛型设计，具有很好的通用性

2. `vector<int> myV = {1, 2, 3};`

反向输出：

```
for (rit = myV.rbegin(); rit != myV.rend();  
     rit++)
```

```
    cout << *rit << ' ';
```

```
cout << '\n';
```

青岛科技大学

3. vector (向量容器)

① 相当于动态数组 (当超过空间大小时通常按两倍大小扩展, 用于存储具有相同数据类型的序列元素;

② 可以在末尾快速的插入与删除元素.

③ 快速地随机访问元素

④ 在序列中间插入、删除元素较慢, 因为需要移动插入或删除后面的所有元素.

4. leet code 215

```
int findKthLargest (vector<int> &nums, int k) {  
    sort (nums.begin(), nums.end());  
    return nums [(int)nums.size() - k];  
}
```

5. Sort Lamda 表达式

① `sort (myv.begin(), myv.end(), [](int a, int b) -> bool { return a < b; })`; 第 3 页

青岛科技大学

② `sort(myv.begin(), myv.end(), [int a, int b) -> bool {return a > b;});`

b. Leetcode (1243)

```
int countSegments (string & s) {  
    int res = 0;  
    for (char x : s) res += (int)(x == ' ');  
    return res + 1;  
}  
int res = 0, n = s.size();  
for (int i = 0; i < n; i++) {  
    if (s[i] != ' ' && s[i+1] == ' ') {  
        res++;  
    }  
}  
return res;  
}
```

三. String 容器

1. 定义方式

① `string()`; ② `string(const string& str);`

③ `string(const string& str, size_type i);`

青岛科技大学

④ `string (const string& str, size_type i, size_type len);`

⑤ `string (const char * cstr);`

⑥ `string (const char * chars, int len)`

⑦ `string (int len, char c);`

第二章 递归算法

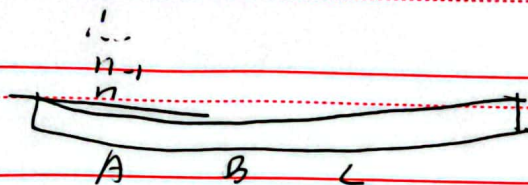
二、递归 一、问题引入

1. 汉诺塔问题

$f(1)=1, f(2)=3, f(3)=7, f(4)=15$ ★

① $n=1$ 时, $f(1)=1$

$n=1$ 时,



将 $(n-1)$ 个移到 A, 将 n 移到 B, 将 $(n-1)$ 移到 B

② 以 $n=4$ 时, A: 4 B: 0 C: 0

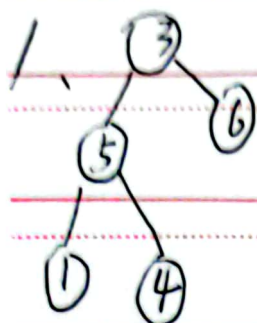
将 3 个移到 C, A: 1 B: 0 C: 3 $f(3)$

将 A 的 1 个移到 B: A: 0 B: 1 C: 3 $f(3)+1$

将 C 移到 B: A: 0 B: 4 C: 0 $f(3)+1+f(3)$

实际递归, $f(n) = 2 \cdot f(n-1) + 1$

青岛科技大学



先: 3 5 1 4 6

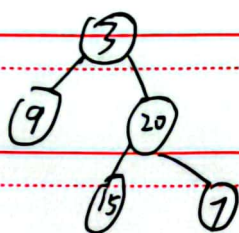
中: 1 5 4 3 6

后: 1 4 5 6 3

2. 构造二叉树

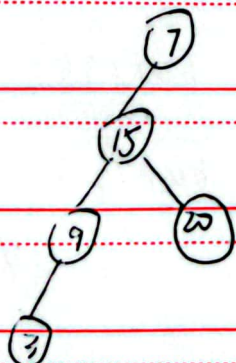
① $pre = \{3, 9, 20, 15, 7\}$

$in = \{9, 3, 15, 20, 7\}$



② 后序 = $\{3, 9, 20, 15, 7\}$

中序 = $\{9, 3, 15, 20, 7\}$



3. 递归求 max-element.

```

int maxv(int l, int r) {
    if (l == r) return a[l];
    return max(maxv(l, mid), maxv(mid+1, r));
}

```

青岛科技大学

二、递归概述

1. 递归模型

① 递归的核心是可以递归模型表示。递归模型是递归算法的抽象，它反映一个递归问题的递归结构。

② 一般地，一个递归模型是由递归出口和递归体两部分组成，递归出口确定递归到何时结束，即指出明确的递归结束条件。

2. 例：给定一个含 n ($n \geq 2$) 个整数的数组

A. 设计一个递归算法求其中最大元素。

解：设最大问题 $f(a, low, high)$ 求 $a[low \dots high]$ 中的最大元素。

$$f(a, low, high) = a[low]$$

$$f(a, low, high) = \max \{ lmaxe, rmaxe \}$$

其中 $lmaxe = f(a, low, mid)$, $rmaxe =$

$$f(a, mid+1, high)$$

青岛科技大学

```
int maxe ( int a [ ], int low, int high ) {  
    if ( low == high )  
        return a [ low ];  
    else {  
        int mid = ( low + high ) / 2 ;  
        int lmaxe = maxe ( a, low, mid );  
        int rmaxe = maxe ( a, mid + 1, high );  
        return max ( lmaxe, rmaxe );  
    }  
}
```

三、递归算法设计方案

1. 第一：数学归纳法原理

若 $\{ p(1), p(2), p(3), p(4), \dots \}$ 是命题序列且满足以下两个性质, 则所有命题为真: (1) 归纳基础: $p(1)$ 为真。且

(2) 归纳步骤: 任何命题均可从前一个命题推出

青岛科技大学

2. 第二类数学归纳法原理: 若 $\{p(1), p(2), p(3), \dots\}$ 是满足以下两个性质的命题序列, 则对于其他自然数, 该命题序列均为真:

(1) 归纳基础: $p(1)$ 为真

(2) 归纳步骤: 任何命题均可以从它的前面所有命题推导得出。

其中归纳步骤的意思是 $p(n)$ 可以从前面所有命题假设 $\{p(1), p(2), \dots, p(n-1)\}$ 推导得出。

3. 直接插入排序

△ △ △ △ △

① 问题描述

对于给定的含有 n 个整数元素的数组 a , 采用直接插入排序法对其元素递增排序, 设计相应的递归排序算法

② 代码设计

青岛科技大学

```
void insertsort (int a[], int n, int i) {  
    if (i > 0) {  
        insertsort (a, n, i-1);  
        if (a[i] < a[i-1]) {  
            int tmp = a[i];  
            int j = i-1;  
            do {  
                a[j+1] = a[j];  
                j--;  
            } while (j >= 0 && a[j] > tmp);  
            a[j+1] = tmp;  
        }  
    }  
}
```

③ 第 ② 法分设

$$T(1) = O(1), \quad T(n) = T(n-1) + O(n)$$

综上. $T(n) = O(n^2)$

青岛科技大学

第3章 穷举法

一、什么是穷举法

① 穷举法，又称枚举法或列举法，其基本思想是先确定有哪些穷举对象和穷举对象的顺序。

② 按穷举对象的顺序逐一列举每个穷举对象的所有情况，再根据问题提出的约束条件验证哪些是可行解。

③ 主要用来解决“是否存在”和“有多少可能性”等类型的问题。

二、例题.

1. 回文子串个数

① 哈希预处理，两层循环枚举判断

② 用 set 去重，枚举起点

③ 中心扩展法

青岛科技大学

2. 最大连续子序列.

brtue Force:

```
int solve ( ) {  
    int max = -998244353;  
    for (int l = 1; l ≤ n; l++) {  
        for (int r = l; r ≤ n; r++) {  
            int s = 0;  
            for (int k = l; k ≤ r; k++) {  
                s += a[k];  
            }  
            max = max (max, s);  
        }  
    }  
    return max;  
}
```

三、回溯 (课外扩展)

1. 线段树维护 lazy tag

① LUOGU P1438 无聊的数列

维护一个数列 a_i , 支持两种操作:

1. $[l, r]$: 给出一个长度为 $r-l+1$ 的等差数列, 首项为 k , 公差为 d , 加到 $[l, r]$ 的 a

2. $2 \text{ } p$: 查询 a_p

青岛科技大学

I. Solution One.

Segment Tree 维护两个 tag: k 与 d

表示在 $[L, r]$ 上加一个首项为 k , 公差为 d 的等差数列.

易发现这两个 tag 均具有叠加性, pushdown

下传懒标记时, 对于 $LC: u \ll 1$: $\begin{cases} K_{LC} + = K_u \\ d_{LC} + = d_u \end{cases}$
对于 $RC: u \ll 1 | 1$: $\begin{cases} K_{RC} + = K_u + (len - LC) * d_u \\ d_{RC} + = d_u \end{cases}$

处理线段树区间询问即可

II. Solution Two

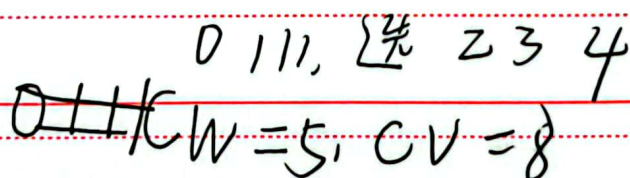
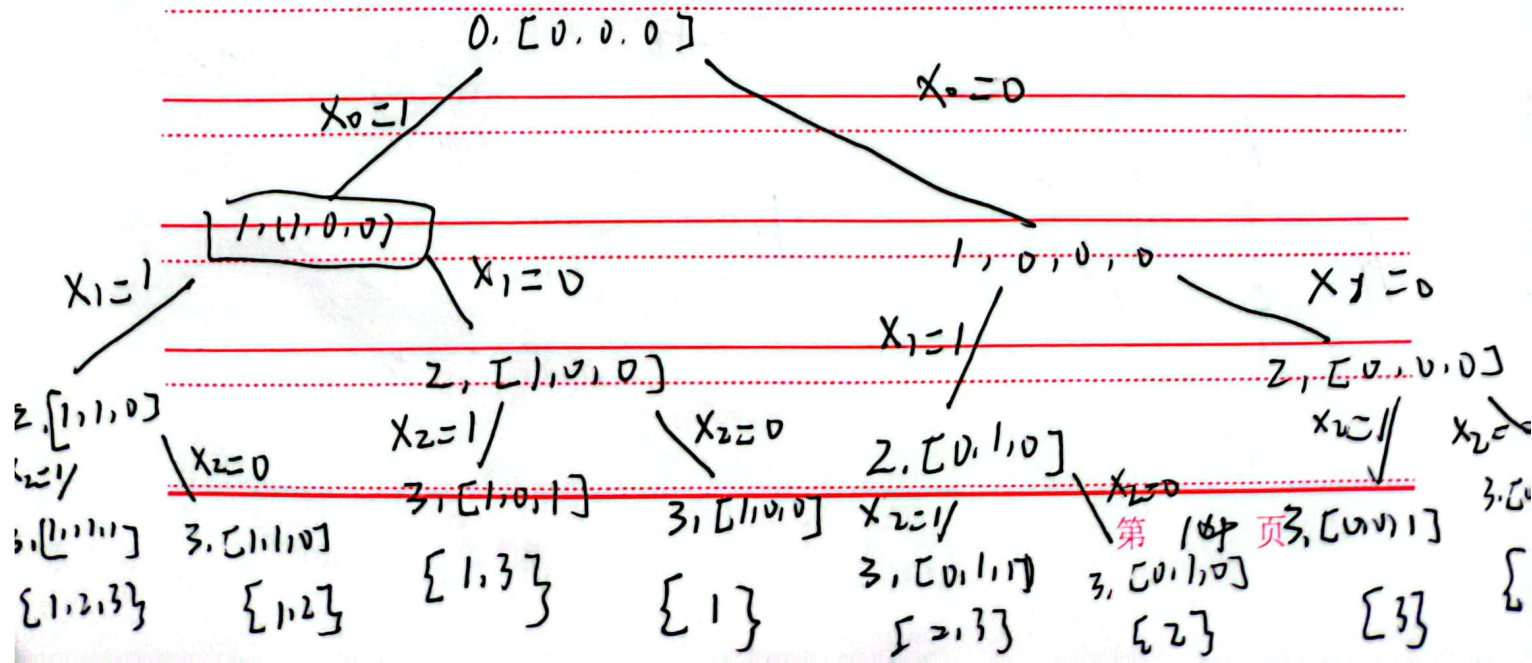
预处理 a_i 为差分数组, 对于操作 1,

$a_L + = k$, $a[L+1, r] + = d$, $a_{r+1} - = d$,

对于操作 2, $answer = \sum_{i=1}^n a_i$

维护最简单的区间加和区间求和即可

$O(n \log n)$


$$0, [0, 0, 0]$$


青岛科技大学

3. 求全排列.

① 问题描述: 给定 $n (n \geq 1)$, 求 $1 \sim n$ 的全排列 pm .

② 代码设计

```
vector<vector<int>> perm2(int n){  
    if (n == 1) return {{1}};  
    vector<vector<int>> pm1 = perm2(n-1);  
    vector<vector<int>> pm;  
    for (auto it = pm1.begin(); it != pm1.end(); it++) {  
        vector<int> e = (*it);  
        for (int j = e.size() - 1; j >= 0; j--) {  
            vector<int> e1 = e;  
            auto it = e1.begin() + j;  
            e1.insert(it, n);  
            pm.push_back(e1);  
        }  
    }  
    return pm;  
}
```

青岛科技大学

第四章 分治法

一、分治法再述基础原理

1. 什么是分治法

① 对于一个规模为 n 的问题, (若容易解决则直接解决。)

② 否则分析问题本身特性, 将其分解成 k ($k \geq 1$) 个规模较小的子问题, 并且在这 k 个子问题互相独立且与原问题类型相同, 然后一一求解子问题, 最后将各子问题解合并得到原问题的解。

③ 这种分而治之 divide & conquer 就叫分治

2. 分治法的特性

① 可缩性 ② 可分解性

③ 可合并性 ④ 独立性

3. 求解过程

① 分解: 将原问题分解为若干个规模较

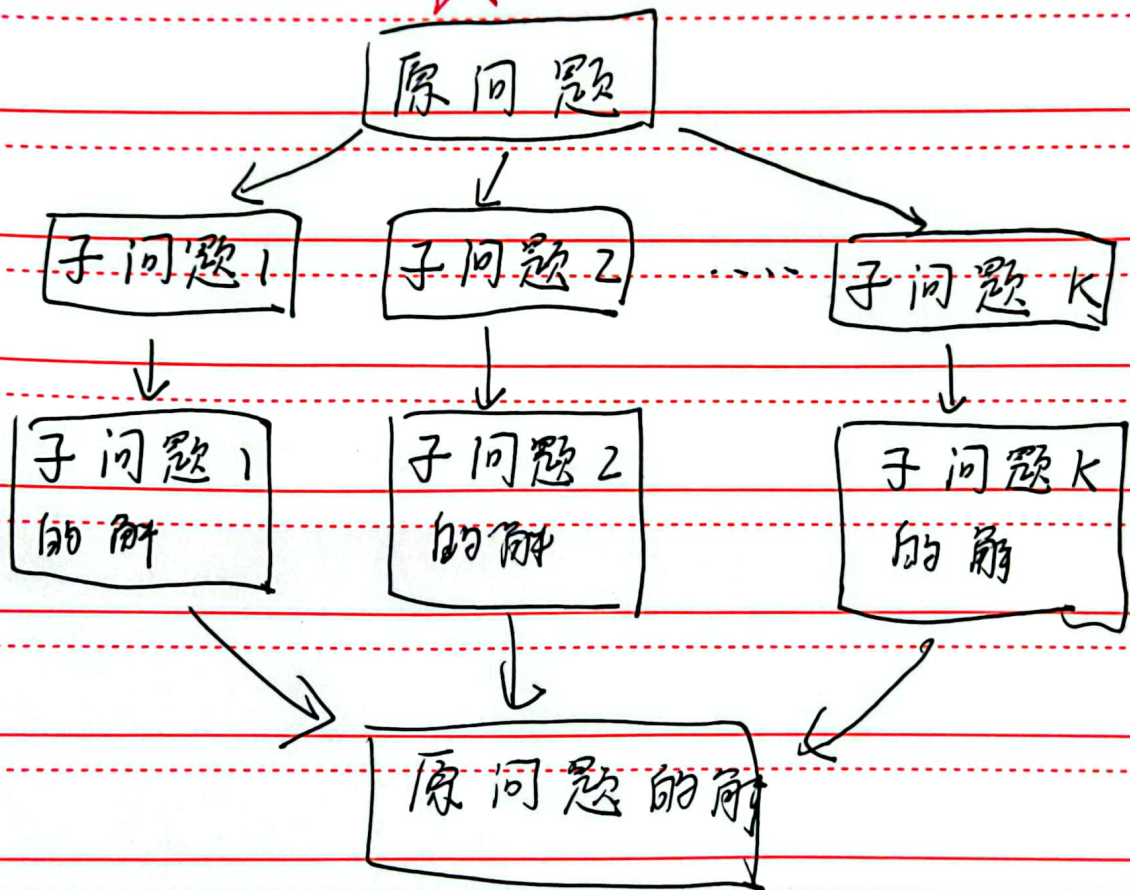
青岛科技大学

小, 相互独立, 与原问题类型相同子问题.

② 求解子问题

③ 合并: 将子问题解合并得到原问题的解

二、图解分治法 



当 $K=1$ 时, 称为减治法 

三、案例引入 一二分查找

青岛科技大学

1. 问题描述

给定一个含 n 个整数的递增有序数组 a , 采用二分查找的方法求整数 k 的元素下标, 若没有找到则返回 -1 .

2. 设 $a[low \sim high]$ 为当前查找区间, 首先确定该区间的中点位置 $mid = \lfloor (low + high) / 2 \rfloor$, 然后将待查的 k 值与 $a[mid]$ 比较:

① 若 $k = a[mid] \Rightarrow$ 查找成功并返回该元素的下标.

② 若 $k < a[mid] \Rightarrow$ 新查找区间改为左子表 $a[low \sim mid - 1]$

③ 若 $k > a[mid] \Rightarrow$ 新查找区间改为右子表 $a[mid + 1 \sim high]$. 下一次查找针对新的查找区间进行.

3. 代码设计

```
① int binsearch(vector<int>&a, int k)
    int n = a.size();
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (k == a[mid])
            return mid;
        if (k < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}
```

② 等价的 基于二分查找递归算法:

```
int binsearch1(vector<int>&a, int
low, int high, int k) {
```

```
if (low <= high) {
```

```
    int mid = (low + high) / 2;
```

```
    if (k == a[mid])
```

```
        return mid;
```

```
    if (k < a[mid])
```

```
        return binsearch1(a, low, mid - 1, k);
```

```
    } else
```

```
        return binsearch1(a, mid + 1, high, k);
```

```
}
```

```
else return -1;
```

```
}
```

```
int binsearch1(vector<int>& a, int k) {
```

```
    int n = a.size();
```

```
    return binsearch1(a, 0, n - 1, k);
```

```
}
```

4. 时间复杂度:
$$\begin{cases} T(n) = 1 & n = 1 \\ T(n) = T(n/2) + 1 & n \geq 2 \end{cases}$$

$\Rightarrow T(n) = O(\log_2 n)$

青岛科技大学

第 五 章

回溯法

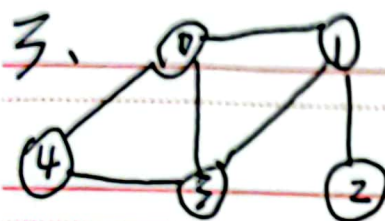
一、基础原理

1. 在解空间中按 DFS 搜索解, 实际上

树的遍历主要有 先根遍历 和 层次遍历.

前者就是 DFS, 后者就是 BFS

2. 回溯法 = DFS + 剪枝



② 邻接矩阵

① 邻接矩阵表

[0] → 1 → 4 → ϕ

[1] → 0 → 2 → ϕ

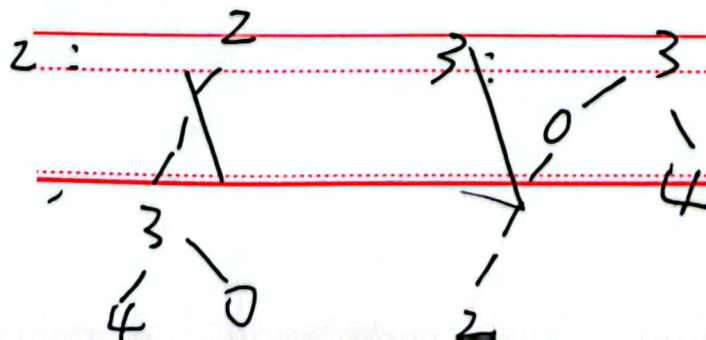
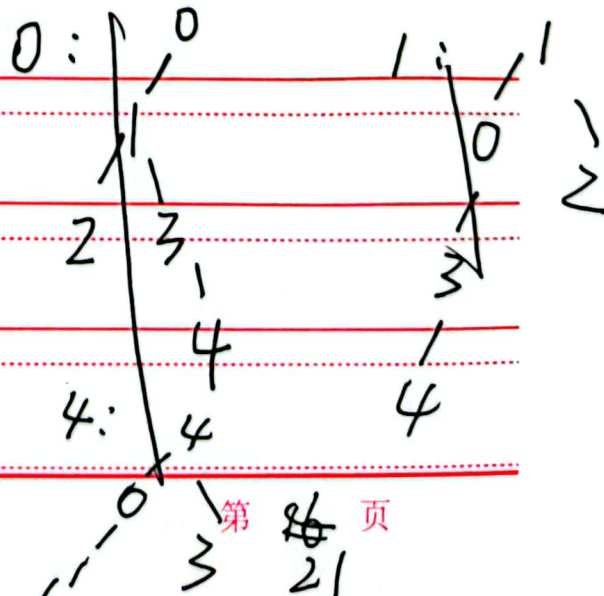
[2] → 1 → ϕ

[3] → 1 → 0 → 4 → ϕ

[4] → 0 → 3 → ϕ

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	0
2	0	1	0	0	0
3	1	1	0	0	1
4	1	0	0	1	0

③ 解空间树 (其中一种)



青島科技大學

二、Codeforce 題目

CF 1904 D2

從左邊 $[1, i-1]$ 找到

一、① 對於 $a_i < b_i$ ，找到 $[j, i]$ 滿足 j ，使得

$a_j = a_i$ ，且 $a_{j+1:n-1} < a_i$ ；且 $b_{j+1:n-1} \geq b_i$

② 從 $[i+1, n]$ 找到 $a_j = b_i$ 且 $a_{i+1:j} < b_i$

且 $b_{j+1:j} \geq b_i$

③ 找相等用 set 維護所有值的下標即可。

④ 區間 mx 和 維護 a 的區間 mx ， b 的區間 mx

用 st 表离线維護即可

二、1. Case One

5

1 2 3 2 4

1 3 3 2 4

1=1,

3=3 4=4

2<3,

向左找到了 3。

但是 4>3。

No.

3. Case Three

5

3 2 1 1 1

3 3 3 2 2

2<3，向右找到 3

所有 $a_i \leq 3$ ， $b_i \geq 3$

3=3 2=2 4=4

2. Case Two

5

3 4 2 2 4

3 4 3 4 4

3=3，2<3，左找到 3

且 $[2, 3] \leq 3$ ， $[3, 3] \geq 3$

1 向左找到了，同理。

第 22 頁

三、Sparse Table, (基于倍增)

1. ST表主要用来解决 RMQ 问题, 比如区间最大 / 最小值查询

2. ① $f_{i,j}$ 维护 $[i, i + 2^j - 1]$ 的信息

② 基于倍增的思想, $O(n \log n)$ 预处理,

$O(1)$ 查询

3. Template

```
int n, q;
```

```
int const N = 1e5 + 10;
```

```
int const SZ = --lg(N) + 2;
```

```
struct SparseTable {
```

```
    int f[N][SZ], g[N][SZ];
```

```
    int Log2[N];
```

```
    void init() {
```

```
        for (int i = 2; i ≤ n; i++)
```

```
            Log2[i] = Log2[i/2] + 1;
```

```
    }
```

```
    for (int i = 1; i ≤ n; i++)
```

```
        f[i][0] = read();
```

```
        for (int j = 1; j < SZ; j++)
```

```
            for (int s = 1; j + (1 << s) - 1 ≤ n; s++)
```

```
                f[i][s] = max(f[i][s-1], f[i + (1 << s) - 1][s-1]);
```

```
int ask(int l,
```

```
int r) {
```

```
    int S = Log2[r - l + 1];
```

```
    return max(
```

```
        f[l][S],
```

```
        f[r - (1 << S) + 1][S]);
```

```
}
```

```
}
```

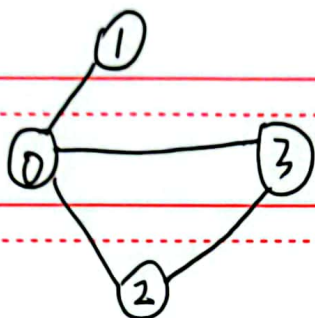
$\max(f[j, j+2^{i-1}-1],$

$f[j+2^{i-1}, j+2^i-1])$

四、图的 m 着色问题.



1. 如图, G 是无向连通图, 相邻点颜色不同. 3 种颜色有 12 种方案



2. 解:

存储: 邻接矩阵 A

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

解向量 $X = \{x_0, x_1, \dots, x_{n-1}\}$.

其中 $X[i]$ 表示顶点 i 的着色

对应的解空间数是一棵 m 叉树, 高度为 $n+1$

i 从 0 开始, 当 $i = n$ 时对应一个叶子结点, $ans++$;

3. 代码设计

```
int n = 4;
```

```
int A[N][N] = { {0,1,1,1}, {1,0,0,0}, {1,0,0,1}, {1,0,0,0} };
```

```
int ans = 0;
```

```
int X[N];
```

```
bool judge (int i, int j) {
```

```
    for (int k = 0; k < n; k++) {
```

```
        if (A[i][k] && X[k] == j)
```

```
            return false;
```

```
    }
```

```
    return true;
```

```
}
```

```
void dfs (int m, int i) {
```

```
    if (i >= nn) ans++;
```

```
    else {
```

```
        for (int j = 0; j < m; j++) {
```

```
            X[i] = j;
```

```
            if (judge(i, j)) dfs(m, i+1);
```

```
            X[i] = -1;
```

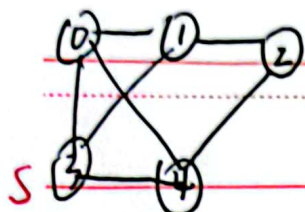
```
        }
```

```
    }
```

```
}
```

第六章 分支界限法 - 基本原理

1. 写出从 S 次点到每个顶点的 最小边数, dist 数组



qu 3 0 1 4 2

dist 0 1 1 1 2

A	0	1	2	3	4
0	0	1	0	1	1
1	1	0	1	1	0
2	0	1	0	0	1
3	1	1	0	0	1
4	1	0	1	1	0

2. 广度优先搜索算法

`vector<int> bfs (vector<vector<int>> &A, int s)`

```

{
    int n = A.size();
    vector<int> dist(n);
    vector<int> vis(n, 0); // 访问标记数组
    queue<int> q; // 队列
    q.push(s);
    vis[s] = 1;
    dist[s] = 0;
    while (q.size()) {
        int u = q.front(); q.pop();
        for (int v = 0; v < n; v++) {
            if (A[u][v] == 1) {
                if (vis[v] == 0) {
                    dist[v] = dist[u] + 1;
                    q.push(v);
                    vis[v] = 1;
                }
            }
        }
    }
    return dist;
}
    
```

青岛科技大学

3. 传送门问题 LinCode 750

① 解法 1: 该问题 Chell 可以上下左右移动, 每移动一步花费的时间都是 1 分钟, 求到达某个位置 E 的最少时间; 每一步代价相同并且求最小总代价, 所以满足利用广度优先搜索求最优解的条件。

② 解法 2: 采用分层次的广度优先搜索方法。广度优先搜索时队列中结点是一层一层地处理的, 首先队列中只有一个根结点, 即第一层结点的个数为 1, 循环一次处理完第一层的全部结点, 同时队列中恰好包含第二层的全部结点。

③ 求出队列中结点个数 cnt , 循环 cnt 次处理完第二层的全部节点, 同时队列中恰好包含第三层的全部结点。

青岛科技大学

③ 解法 3: 采用多起点多层次的广度优先搜索方法。所谓多起点广度优先搜索就是先将多个起始点进队, 然后按基本广度优先搜索找目标点。如果改为分层次优先搜索找目标点, 就称为多起点分层次的广度优先搜索。

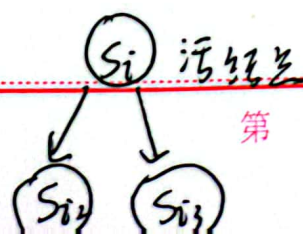
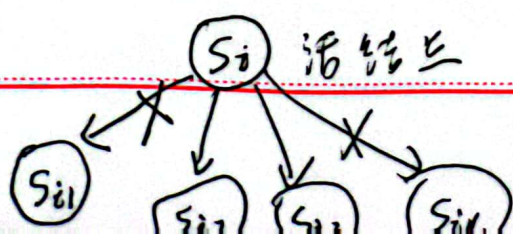
4. 队列式分支限界法框架

① 在解空间中搜索解时, 队列式分支限界法与广度优先搜索一样都是采用队列存储活结点, 从根节点开始一层一层地扩展和搜索结点;

② 同时采用剪枝提高搜索的性能。

4. 分支限界法的时间要点

① 设计合适的限界函数



② 限界函数的特性

⇒ 假设解向量 $X = (x_0, x_1, \dots, x_{n-1})$,

如果目标函数是求最大值。

I. 设计上界限函数 $ub()$,

若从 x_i 的分支向下搜索

所得到的部分解是 $(x_0,$

$x_1, \dots, x_i, \dots, x_k)$, 则

应接满足: $ub(x_i) \geq$

$ub(x_{i+1}) \geq \dots \geq ub(x_k)$.

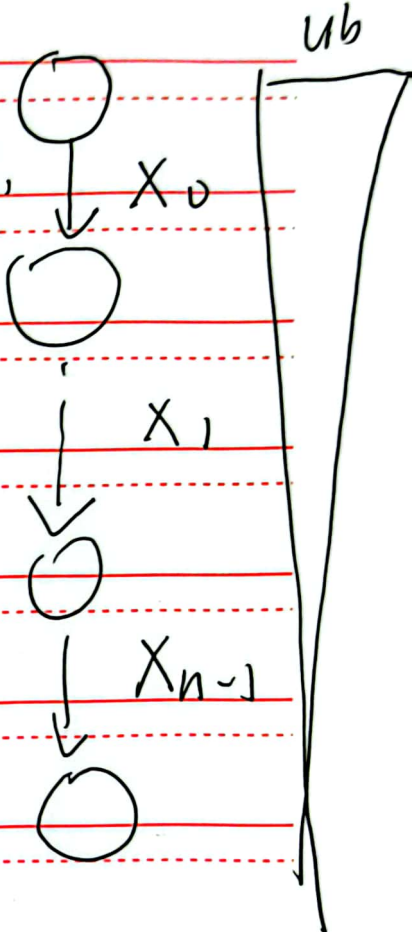
II. 所以根结点 ub 值应该大

于或等于最优解的 ub 值。如果从

S_i 结点扩展到 S_j 结点, 应满足

$ub(S_i) \geq ub(S_j)$, 将所有小于

$ub(S_i)$ 的结点剪支。



⇒ 假设解向量 $X = (x_0, x_1, \dots, x_{n-1})$,

如果目标函数是求最小值。

I. 设计下界函数 $lb()$ 。

若从 α_i 分支向下搜索

所得到的部分解是 $(\alpha_0,$

$\alpha_1, \dots, \alpha_i, \dots, \alpha_k)$ ，则

应满足： $lb(\alpha_i) \leq lb$

$(\alpha_{i+1}) \leq \dots \leq lb(\alpha_k)$

II. 所以从根结点的 lb 应

小于等于最优解的 lb 值。

如果从 S_i 结点扩展到

S_j 结点，应满足 $lb(S_i)$

$\leq lb(S_j)$ ，将所有大于

$lb(S_i)$ 的结点剪支。

